

Forge-Q

Certified Quantitative and Relational Proof Search

A concrete extension of Forge for probabilities,
expected costs, couplings, and randomized programs

Prepared for Vladimir Reshetnikov
September 15, 2026

Deliverables: six exact-rational Python workers, separate certificate checking, independent small-instance oracles, a reproducible corpus, candidate Lean foundations, and implementation contracts.

Evidence boundary: 520 accepted certificate records; 520 rejected targeted mutations; 24 unit-test methods; five explicit inconclusive controls. The Lean candidates were *not compiled*: no Lean or Lake executable was available. This package does not install a Lean tactic and establishes no speedup over an existing tactic.

Forge baseline: `c98e47c5f804e92880fc1d0e378c1b95832b685c`.

Abstract

Forge already combines structural proof planning with exact algebraic certificates, observable-space and invariant-ideal closures, finite covers, and semantic countermodels. Repackaging those mechanisms would not advance the project. This article develops a separate quantitative branch: maximum reachability in rational Markov decision processes, expected hitting-time bounds valid for every scheduler, exact finite couplings and transport duals, certified probabilistic-bisimulation refinement, quantitative coupling bounds valid at every time horizon, and symbolic potential synthesis for infinite-state random walks.

The central change is not a larger collection of arithmetic lemmas. It is a family of proof rules converting finite, locally checkable inequalities and couplings into statements about unbounded probabilistic behavior. The treatment distinguishes a Bellman fixed point from its intended least solution, bisimulation failure from trace inequivalence, a failed contraction rate from nonmixing, and finite expected cost from termination. Search and checking are implemented separately using exact rational arithmetic. Every recorded case has an independent small-instance oracle or finite-distribution cross-check; their scope and limitations are explicit. The underlying mathematical theories are established prior art. The contribution is their concrete, compositional specialization into new Forge workers, including executable certificate formats and a falsifiable Lean integration plan.

Contents

1	The actual delta from the inspected repository	2
2	Semantic foundations and the finite proof boundary	3
3	Worker I: exact maximum reachability with progress	5
4	Worker II: expected costs under every scheduler	7
5	Worker III: exact couplings, optimal transport, and Hall obstructions	9
6	Worker IV: certified probabilistic-bisimulation refinement	11
7	Worker V: all-horizon coupling and perturbation bounds	13
8	Worker VI: symbolic potentials for infinite-state walks	14
9	Lean integration: concrete modules and acceptance obligations	17
10	Implementation, formats, and resource accounting	20
11	Recorded experiments and their limits	22
12	Development priorities and assessment	24
A	Reproduction and artifact map	25
B	Primary references and source pins	26

1 The actual delta from the inspected repository

1.1 What is being extended, not reinvented

The inspected Forge revision is

c98e47c5f804e92880fc1d0e378c1b95832b685c.

Its README describes an 111-page synthesis of eighteen proposals, with a design round and an extension round. It explicitly says that `forge` is not an implemented Lean tactic. The merge subsequently elaborated some core-only Lean specimens; this is different from having a Lean-verified certificate checker or a tactic benchmark [1, 4].

The present review used the root README, repository tree, the neighboring-project review, the conclusion, the provenance index, the observable-closure part of Section 6, and the opening nonlinear-certificate sections. Leant’s and Djex’s root descriptions were also read. Connector searches for coupling and probabilistic material returned no matches. This is a scoped design audit, not a claim to have reread every frozen submission or rerun the repository. The overlap assessment rests primarily on the merged capability descriptions and provenance index, rather than on a negative search result [2, 5].

Existing Forge mechanism	Deliberately reused	Addition in this article
Certificate-producing outer planner	Scope binding, evidence discipline, local arithmetic checking	Quantitative goal classification with explicit scheduler and horizon semantics
Weighted-word observable closure	Exact trace-distribution equality after a justified encoding	Reachability extrema and inequalities; nonlinear transport optimization
Target-generated invariant ideals	Pathwise polynomial identities	Expectation operators, which are linear but generally not ring homomorphisms
Finite-algebra covers	Finite closure infrastructure	Relations closed under <i>coupling</i> , with Hall-certified deletions
Cone, quadratic, Bernstein, and Sturm workers	Polynomial positivity subgoals	Symbolic drift and cost obligations generated from probabilistic transitions
Typed synthesis and Leant/Djex boundaries	Exact candidate identity and original-type checking	Attaching distributional equivalence and cost evidence to reflected randomized candidates

1.2 Why this is a substantial capability increase

The new questions are not merely equalities between finitely many samples. They include

$$\sup_{\sigma} \mathbb{P}_s^{\sigma}(\text{eventually reach } B) = r, \quad \forall \sigma, \quad \mathbb{E}_s^{\sigma} T_B \leq V(s),$$

and

$$\forall k \in \mathbb{N}, \quad W_d(\mu P^k, \nu Q^k) \leq \kappa^k W_d(\mu, \nu) + \varepsilon \frac{1 - \kappa^k}{1 - \kappa}.$$

Here σ resolves nondeterministic choices, T_B is a hitting time, and W_d is finite optimal transport for a specified metric. A coupling is itself a useful existential witness: a joint distribution with prescribed marginals. Producing one is an algorithmic task that ordinary equality saturation does not perform merely by increasing its fuel.

The contribution is *relative to Forge*, not a claim that probabilistic model checking, coupling, or ranking supermartingales are new. Quantitative reachability certificates have an established literature; probabilistic product programs and expected-sensitivity logics provide the appropriate relational precedents; martingale-based program analysis supplies the cost perspective [8, 9, 10, 11]. This article gives the certificate equations and their proofs directly so that the engineering proposal does not depend on trusting those references as a black box.

1.3 Four examples that force different proof mechanisms

A retry loop that succeeds with probability $1/2$ has expected duration 2, although no finite unrolling proves that every run stops. A nondeterministic retry loop with success probabilities $1/2$ and $1/4$ has expected duration at most 4 under every scheduler; selecting the faster action only proves a weaker statement.

A system may reach success with probability $1/2$ and otherwise enter a closed failure state. Its reachability probability is still exactly certifiable, but its expected time to *success* is infinite. The auxiliary rank for reaching either success or failure must not be advertised as a success-time bound.

Two randomized programs can have equal observable trace laws without admitting strong probabilistic bisimulation. A failed bisimulation search therefore cannot reject a trace-equivalence goal. Conversely, a supported coupling may directly prove a relational goal even when no state-by-state syntactic correspondence is obvious.

Finally, for a walk that moves down with probability $2/3$ and up with probability $1/3$, stopped at zero, the potential

$$V(n) = \frac{3}{2}n^2 + \frac{9}{2}n$$

proves an upper bound on the expected accumulated state value $\mathbb{E} \sum_{t < T} X_t$. This is an infinite-state result obtained by checking a polynomial identity and signs, not by truncating the state space.

2 Semantic foundations and the finite proof boundary

2.1 Rational kernels and named quantifiers

Let S be a finite nonempty state set. A transition kernel is a matrix $P \in \mathbb{Q}^{S \times S}$ with $P(s, t) \geq 0$ and $\sum_t P(s, t) = 1$. A finite Markov decision process, or MDP, gives a nonempty finite set $A(s)$ of such rows at each state. All entries denote mathematical rationals, not floating-point estimates.

A scheduler may depend on the complete finite execution history and may randomize among enabled actions. We state bounds for this full scheduler class whenever all-action inequalities are checked. A selected stationary deterministic policy $\pi(s) \in A(s)$ is used only when the certificate explicitly supplies an existential policy witness. Erasing this distinction changes the proposition.

For a target set B , the implementation requires target states to be absorbing. An arbitrary source program can be converted to this stopped model for *first-hit* questions, but only after proving that this conversion preserves the requested event or accumulated pre-hit cost. It does not preserve

arbitrary post-hit behavior. Empty target sets are mathematically allowed; the test generators use nonempty ones.

2.2 The expectation transformer

For a rational vector f , define

$$(P_a f)(s) = \sum_t P(s, a, t) f(t).$$

Nonnegative rows make this operator monotone. The maximum finite-horizon hitting probabilities are generated by

$$\begin{aligned} H_0(s) &= \mathbf{1}_B(s), \\ H_{k+1}(s) &= \begin{cases} 1, & s \in B, \\ \max_{a \in A(s)} P_a H_k(s), & s \notin B. \end{cases} \end{aligned}$$

The infinite-horizon value is $H(s) = \sup_k H_k(s)$. This equals the supremum over all schedulers of eventual-hit probability: each bounded horizon has a finite backward-induction optimizer, and both sides are suprema over the same finite-horizon probabilities. No claim about finding one policy optimal for all horizons is needed for this equality.

For a nonnegative cost c that is zero on B , the corresponding finite-horizon maximum accumulated costs satisfy $C_0 = 0$ and

$$C_{k+1}(s) = c(s) + \max_{a \in A(s)} P_a C_k(s) \quad (s \notin B), \quad C_{k+1}|_B = 0.$$

The same recurrences can be formulated using nonnegative extended reals. Finite rational bounds are then transported into that semantic domain. This avoids silently assuming integrability of the very random variable whose finite expectation is being proved.

Lemma 2.1 (Supersolutions bound every finite horizon). *Let F be a monotone operator on an ordered space. If $x_0 \leq V$ and $F(V) \leq V$, then $F^k(x_0) \leq V$ for every $k \in \mathbb{N}$.*

Proof. The base is the hypothesis. If $F^k(x_0) \leq V$, monotonicity gives $F^{k+1}(x_0) \leq F(V) \leq V$. $\square$

This short induction is the first Lean target. The substantial semantic work is showing that the reflected source program has precisely the operator F in the certificate. Once that bridge is proved, the checker need not solve a stochastic process again.

2.3 What the external problem contains

The Python checker receives the problem and certificate separately. For an MDP, the problem contains every action row and the target set. For transport, it contains both marginals, the allowed relation, and the cost matrix. For contraction, it additionally contains the full metric, requested rate, and error. For a symbolic walk, it contains the jump support, probabilities, and requested cost polynomial.

A certificate is not allowed to replace any of these inputs. A future Lean reifier must bind them to the original expression and context. This exact-source discipline already belongs to Forge; the new requirement is *which probabilistic data* must be bound, including the scheduler quantifier, stopping boundary, observations, and whether a random choice is fresh at each step.

3 Worker I: exact maximum reachability with progress

Status. Implemented and tested in Python. The complete Lean checker and infinite-horizon source theorem are not implemented.

3.1 The fixed-point trap

A non-target self-loop satisfies $v(s) = Pv(s)$ for every number $v(s)$. Thus the equation $v = Pv$ does not determine reachability. In particular, accepting $v(s) = 1$ for a state that never reaches the target would be unsound. A least-fixed-point problem cannot be solved by checking arbitrary fixed-point equations.

A Bellman *upper* vector needs no progress: monotone iteration begins below it. An exact value needs an additional argument that a selected policy actually achieves it. The certificate below supplies that argument through a finite exit-time potential.

3.2 Certificate equations

A certificate consists of a value vector v , a stationary policy π , a set D of policy-closed zero-value states, and a nonnegative rank vector r . Write $U = S \setminus (B \cup D)$. Check

$$0 \leq v \leq 1, \quad v|_B = 1, \quad v|_D = 0, \quad D \cap B = \emptyset, \quad (1)$$

$$P_a v(s) \leq v(s) \quad (s \notin B, a \in A(s)), \quad (2)$$

$$P_\pi v(s) = v(s) \quad (s \notin B), \quad (3)$$

$$P_\pi(s, t) = 0 \quad (s \in D, t \notin D), \quad (4)$$

$$r \geq 0, \quad r|_{B \cup D} = 0, \quad r(s) \geq 1 + \sum_{t \in U} P_\pi(s, t)r(t) \quad (s \in U). \quad (5)$$

The checker verifies row normalization, dimensions, policy indices, boundary sets, and every displayed rational equality or inequality. It does not compute graph reachability, solve a linear system, or trust a producer-supplied residual.

Theorem 3.1 (Exact reachability certificate). *Equations (1)–(5) imply*

$$v(s) = \sup_{\sigma} \mathbb{P}_s^{\sigma}(T_B < \infty) = \mathbb{P}_s^{\pi}(T_B < \infty) \quad \text{for every } s.$$

Proof. The all-action inequality and Lemma 2.1 give $H_k \leq v$, hence $H \leq v$.

Under the selected policy let τ be the first entry into $B \cup D$. Apply the same finite-horizon induction to the killed kernel on U and unit cost there. Equation (5) gives $\mathbb{E}_s(\tau \wedge k) \leq r(s)$ for every k . Therefore τ is finite almost surely, and $\mathbb{P}_s(\tau > k) \leq r(s)/k$ for $k > 0$.

The boundary values and harmonic equations give the finite identity

$$v(s) = \mathbb{P}_s^{\pi}(T_B \leq k) + \mathbb{E}_s^{\pi}[v(X_k)\mathbf{1}_{\{\tau > k\}}].$$

It follows either by induction or by decomposing stopped paths. The second term is between zero and $\mathbb{P}_s(\tau > k)$, because $0 \leq v \leq 1$. It vanishes as k grows. Thus v is the selected policy's hitting probability. Since the selected policy is one permissible scheduler, $v \leq H$, completing the proof. $\square$

3.3 An exact reference producer

The supplied producer enumerates stationary deterministic policies. For each policy it computes, by reverse positive-edge reachability, the states from which B is graph-reachable. Their complement is D . On U it solves

$$(I - P_{UU})v_U = P_{UB}\mathbf{1}, \quad (I - P_{UU})r_U = \mathbf{1}.$$

Every state of U has a positive-probability path to B , and there is no closed recurrent class confined to U in a finite chain. More concretely, one can choose finitely many paths and obtain a common positive probability of leaving U within a bounded number of steps. Hence the two systems are nonsingular and the resulting exit expectations are finite. The producer then tests every action against v and returns the first qualifying certificate.

```

for each stationary deterministic policy pi:
  D := states with no positive-edge path to target under pi
  U := states outside target and D
  solve exactly (I - P[U,U]) * v = P[U,target] * 1
  solve exactly (I - P[U,U]) * r = 1
  if every action satisfies P_a v <= v:
    return (v, pi, D, r)
return UNKNOWN if the policy budget was exhausted

```

Proposition 3.2 (Existence of a certifiable optimal policy). *For every finite rational MDP, a certificate of the preceding form exists.*

Proof. Let $h = \sup_k H_k$. A finite maximum and finite weighted sums commute with this increasing limit, so h satisfies the Bellman equation. Call an action greedy when $P_a h(s) = h(s)$, and draw every positive-probability edge of every greedy action.

Every state with $h(s) > 0$ has a greedy-graph path to B . Otherwise let C be the positive-value states without such a path. A greedy transition from C can leave C only toward a zero-value state. Replacing h by $(1 - \delta)h$ on C therefore preserves the greedy supersolution inequalities there. Each nongreedy action has a strictly positive slack; finiteness lets us choose a common sufficiently small $\delta > 0$ preserving those inequalities too. Outside C , only successor values were decreased, so the supersolution inequalities remain true. This constructs a nonnegative upper vector $w < h$ somewhere with $w|_B = 1$, contradicting $H_k \leq w$ and hence $h \leq w$.

At each positive non-target state select a greedy action containing an edge that decreases the shortest greedy-graph distance to B . No closed class can remain wholly among positive non-target states: a state of minimum such distance in that class has a positive selected edge leaving it. The selected policy thus exits to B or to the zero-value region almost surely with finite mean exit time. That zero region is closed under every action, since nonnegative expectations of h there are zero. The finite transient linear systems yield rational h and a rational exit rank, giving the desired certificate. $\square$

This is the standard finite policy-optimization setting underlying reachability-certificate work [8], with the properness step made explicit for replay. Consequently the unrestricted reference enumeration is complete for this finite rational fragment. The implementation has a policy cap and size limits, so its operational answer may be UNKNOWN. No polynomial-time claim is made for enumeration: if $a_s = |A(s)|$, up to $\prod_s a_s$ policies are tried. Each dense policy solve uses $O(|S|^3)$

rational operations. The checker needs only a pass over the supplied transitions and vectors; bit complexity still depends on numerator and denominator sizes.

Production search should use a model checker or policy-iteration/LP backend to propose a policy, then recompute its certificate exactly. Choosing an action that merely maximizes an approximate value can select a spurious self-loop among tied actions. The certificate repairs this risk only if the proposed policy passes the harmonic *and progress* conditions. A failed proposal must trigger another search, not a weakened check.

3.4 A nontrivial worked certificate

Use states (s, d, b) , with d a failure sink and b a success sink. At s , the two rows are

$$P_0(s, \cdot) = (1/2, 1/4, 1/4), \quad P_1(s, \cdot) = (0, 2/3, 1/3).$$

Choose action zero, $D = \{d\}$,

$$v = (1/2, 0, 1), \quad r = (2, 0, 0).$$

The first action has expected value $\frac{1}{2} \cdot \frac{1}{2} + \frac{1}{4} = \frac{1}{2}$, the second has expected value $\frac{1}{3}$, and the rank equation is $2 = 1 + \frac{1}{2} \cdot 2$. Thus the best success probability is exactly $1/2$.

The rank bounds the time until *success or failure*. It does not bound T_B : failure has positive probability. This example is a useful source-bridge regression because confusing the two stopping sets turns a correct certificate into a false theorem.

3.5 Minimum reachability is deliberately not obtained by flipping signs

Replacing every upper inequality by a lower inequality would reintroduce spurious fixed points. A minimum-reachability worker needs its own greatest/least-solution and scheduler argument, or a separately verified dual characterization. This package implements only maximum reachability. Exact certificates for both extrema exist in prior work [8]; incorporating another format is a future worker, not an implicit consequence of the present code.

4 Worker II: expected costs under every scheduler

Status. Uniform unit-cost hitting-time search and checking are implemented. General nonnegative cost is proved below as an extension contract; the symbolic worker implements polynomial costs separately.

4.1 A local certificate for an unbounded sum

Let $c(s) \geq 0$ be zero on the absorbing target B . A nonnegative potential V with $V|_B = 0$ is a uniform cost certificate when

$$V(s) \geq c(s) + \sum_t P(s, a, t) V(t) \quad \text{for every } s \notin B \text{ and every } a \in A(s). \quad (6)$$

Theorem 4.1 (Uniform expected cost). *If (6) holds, every history-dependent randomized scheduler satisfies*

$$\mathbb{E}_s^\sigma \sum_{t=0}^{T_B-1} c(X_t) \leq V(s).$$

If additionally $c(s) \geq \varepsilon > 0$ outside B , then

$$\mathbb{E}_s^\sigma T_B \leq V(s)/\varepsilon, \quad \mathbb{P}_s^\sigma(T_B < \infty) = 1.$$

Proof. The all-action inequality remains true after a scheduler takes a convex combination of actions, conditional on any finite history. Telescoping for the first k stopped steps gives

$$\mathbb{E}_s^\sigma \left[\sum_{t < k \wedge T_B} c(X_t) + V(X_{k \wedge T_B}) \right] \leq V(s).$$

Drop the nonnegative terminal potential. The partial accumulated costs increase monotonically, so their extended expectations converge to the expected total cost. This proves the first assertion without assuming that T_B is integrable. The lower bound by $\varepsilon(k \wedge T_B)$ yields the second assertion by the same limit. A finite expectation rules out positive probability of $T_B = \infty$. $\square$

The proof is elementary ranking-supermartingale reasoning, formulated to avoid circular applications of optional stopping [11]. The finite-horizon identity is the object to formalize first; the infinite-horizon theorem is a separate semantic bridge.

4.2 Search and useful negative controls

The reference producer is intentionally small. It enumerates policies, solves $(I - P_{UU})V_U = \mathbf{1}$ with $U = S \setminus B$, rejects singular or negative candidates, and checks *all* enabled actions against the resulting vector. This produces a uniform bound, not just the expectation under the enumerated policy. We do not use an unsuccessful enumeration as a certificate of infinite expectation.

For a single retry state with actions having self-loop probabilities $1/2$ and $3/4$, the selected worst-case potential is $V = 4$:

$$1 + \frac{1}{2} \cdot 4 = 3 \leq 4, \quad 1 + \frac{3}{4} \cdot 4 = 4.$$

A potential 2 obtained from the fast action fails the second check. If an action is changed to a probability-one self-loop, the maximum reachability remains one because the immediate-success action is available, but there is no finite uniform unit-cost potential. The test suite keeps this scheduler-polarity control.

With general cost, a zero-cost nonterminating loop satisfies $V = 0$. Its accepted claim is only a zero accumulated-cost bound, never termination. Positive drift in a ranking argument means a positive *expected decrease of the potential*, not merely a nonnegative cost annotation.

4.3 Tail bounds and compositional use

The expected-time certificate already yields

$$\mathbb{P}_s^\sigma(T_B > k) \leq \frac{V(s)}{\varepsilon k}, \quad k > 0.$$

This is a conservative tail estimate, not an exponential bound. A multiplicative drift or exponential-potential certificate could support stronger tails, but that is a different schema and is not implemented here.

For sequential programs, a verified postcondition can provide an upper bound on the next component's potential. If component one has expected local cost at most $V_1(s)$ and its output distribution satisfies $\mathbb{E}V_2(\text{output}) \leq W(s)$, the combined cost is bounded by $V_1(s) + W(s)$. Merely adding two initial-state bounds can be wrong because the second initial state is random. This is where Forge's intermediate-obligation mechanism should request a proved expectation bound rather than treating costs as syntactic annotations.

5 Worker III: exact couplings, optimal transport, and Hall obstructions

Status. Exact rational search, primal-dual checking, and negative Hall certificates are implemented.

5.1 The constructive witness

For probability vectors $\mu \in \mathbb{Q}^I$ and $\nu \in \mathbb{Q}^J$, a coupling is a nonnegative matrix γ satisfying

$$\sum_j \gamma_{ij} = \mu_i, \quad \sum_i \gamma_{ij} = \nu_j. \quad (7)$$

A required relation $R \subseteq I \times J$ is enforced by $\gamma_{ij} = 0$ outside R . A finite cost matrix d adds the objective

$$\langle d, \gamma \rangle = \sum_{ij} d_{ij} \gamma_{ij}.$$

The supplied transport worker restricts d to nonnegative rational entries. Dual potentials may have either sign.

The output is not a statement that an optimizer reported success. It is the joint matrix itself, which can be rendered as a finite distribution or used to construct a coupled sampling program. The elementary marginals proof is the local component of probabilistic product-program reasoning [9].

5.2 Optimality with a tiny checker

Supply dual vectors $u \in \mathbb{Q}^I$ and $z \in \mathbb{Q}^J$ satisfying

$$u_i + z_j \leq d_{ij} \quad ((i, j) \in R) \quad (8)$$

and the equality

$$\sum_{ij} d_{ij} \gamma_{ij} = \sum_i \mu_i u_i + \sum_j \nu_j z_j. \quad (9)$$

Theorem 5.1 (Transport certificate). *A nonnegative supported matrix satisfying (7), together with (8) and (9), is an optimal coupling for the named marginals, relation, and cost.*

Proof. For any feasible coupling η , multiply (8) by η_{ij} and sum. The marginal equations give

$$\langle d, \eta \rangle \geq \sum_i \mu_i u_i + \sum_j \nu_j z_j = \langle d, \gamma \rangle.$$

Only finite sums, nonnegative multiplication, and the supplied identities are used. $\square$

No strong-duality theorem is required by the checker. Strong duality helps justify that complete search can find such witnesses, but acceptance depends only on weak duality plus an explicitly zero gap.

Example 5.2 (A nonzero optimum). Take $\mu = (1/2, 1/2)$, $\nu = (1/3, 2/3)$, the full relation, and the discrete cost $d_{ij} = \mathbf{1}_{i \neq j}$. One optimal joint matrix and dual pair are

$$\gamma = \begin{pmatrix} 1/3 & 1/6 \\ 0 & 1/2 \end{pmatrix}, \quad u = (1, 0), \quad z = (-1, 0).$$

Both primal and dual objective values are $1/6$. The negative dual coordinate is essential; forbidding it would unnecessarily restrict valid certificates.

5.3 A checkable failure of support compatibility

Let $N_R(X) = \{j : \exists i \in X, (i, j) \in R\}$. A Hall obstruction is a subset $X \subseteq I$ satisfying

$$\mu(X) > \nu(N_R(X)). \tag{10}$$

Proposition 5.3 (Hall obstruction soundness). *Equation (10) rules out any coupling supported in R .*

Proof. All mass from X must go to $N_R(X)$, so any supported joint would satisfy

$$\mu(X) = \sum_{i \in X, j \in N_R(X)} \gamma_{ij} \leq \sum_{i \in I, j \in N_R(X)} \gamma_{ij} = \nu(N_R(X)),$$

a contradiction. $\square$

The checker recomputes the *entire* neighborhood from the externally supplied relation. It rejects a supplied list omitting a neighbor, even when the resulting false deficit looks convincing. The result means “no coupling in this relation.” It says nothing about a larger relation or a different relational proof rule.

For finite rational distributions, the bipartite flow construction also establishes completeness: if full unit flow cannot be sent, a residual cut provides such an X . This is an exact infeasibility outcome, unlike exhaustion of a search budget.

5.4 The implemented flow algorithm

Build a network with source-to-left capacities μ_i , left-to-right edges for R with cost d_{ij} , and right-to-sink capacities ν_j . Internal capacities are set to 2, strictly above the total transported mass 1. This avoids accidental saturation constraints contaminating the simple transport dual.

The producer repeatedly finds a shortest residual source–sink path using Bellman–Ford, augments by the exact rational bottleneck, and updates reverse edges. Initial costs are nonnegative; shortest augmentation maintains minimum cost at the current flow value. If the sink becomes unreachable before flow one, residual reachability yields the Hall cut. After full flow, a zero-cost super-source potential computation on the residual graph yields the dual coordinates.

```

flow := 0
while flow < 1:
  shortest_path := BellmanFord(residual_network)
  if sink unreachable:
    X := reachable left vertices
    return Hall(X, recomputed_neighborhood(X))
  augment by the exact residual bottleneck
potentials := shortest potentials from a zero-cost super-source
return joint matrix and transport dual potentials

```

After clearing input denominators, each augmentation sends an integral positive amount in the scaled network, so the unrestricted procedure terminates. This is a pseudopolynomial-style termination argument, not a strong polynomial bound in the input bit length. The implementation has an augmentation budget and uses dense arrays; it is a reference producer. A production backend can use sparse min-cost flow or exact LP without changing the certificate theorem.

The checker costs $O(|I||J|)$ rational operations for signs, support, marginals, dual feasibility, and the two objectives. It does not run Bellman–Ford and does not import any producer module.

6 Worker IV: certified probabilistic-bisimulation refinement

Status. Positive coupling relations, negative deletion traces, and an independent partition-refinement oracle are implemented for finite labeled Markov chains. Nondeterministic probabilistic automata are outside this implementation.

6.1 Greatest-fixed-point search with proof-producing deletions

Let P and Q be finite kernels, with observation maps o_P and o_Q . Define the operator

$$\Phi(R) = \{(s, t) : o_P(s) = o_Q(t) \text{ and } P(s, \cdot), Q(t, \cdot) \text{ admit a coupling supported in } R\}.$$

A relation $R \subseteq \Phi(R)$ is a strong probabilistic bisimulation relation in the relational-lifting presentation. This is a standard probabilistic process-equivalence setting [12].

Start from $R_0 = \{(s, t) : o_P(s) = o_Q(t)\}$. For each candidate pair, ask the transport worker for a zero-cost supported coupling. If it returns a Hall cut, delete that pair and record the cut against the *current* relation. Repeat until no pair is deleted or the queried start pair has disappeared.

The distinction between a simultaneous and sequential iteration matters in replay. This implementation records sequential deletions; each cut is checked relative to the relation after all earlier deletions and before its own. A checker cannot validate every cut against the final, smaller relation: that would allow a circular set of deletions to justify itself.

6.2 Positive and negative certificates

A positive certificate contains R , the queried initial pair, observation compatibility, and one joint matrix for every pair in R . The matrix for (s, t) has marginals $P(s, \cdot)$ and $Q(t, \cdot)$ and support contained in R . The initial pair must belong to R .

A negative certificate is a sequence

$$((s_1, t_1), X_1), \dots, ((s_k, t_k), X_k),$$

where each X_i is a valid Hall obstruction against the relation before deleting (s_i, t_i) . The queried initial pair is absent at the end. If its observations differ initially, the empty sequence is already a valid negative certificate; the checker explicitly distinguishes that case from an unjustified empty trace.

Theorem 6.1 (Certified relation search). *The positive certificate establishes a strong bisimulation containing the start pair. The negative certificate establishes that no strong bisimulation contains that pair. Without operational cutoffs, the finite refinement procedure decides this relation-lifting problem.*

Proof. The positive claim is exactly the postfixed-point condition. For the negative claim, let S be any postfixed relation. Initially $S \subseteq R_0$. If $S \subseteq R_i$ and a Hall cut forbids a coupling for (s, t) supported in R_i , then (s, t) cannot belong to S : a coupling supported in S would also be supported in R_i . Hence S survives every deletion as a subset. An absent start pair cannot belong to any such S .

There are at most $|P||Q|$ deletions. At a pass with none, every surviving pair has a checked coupling in the current relation. Thus the final relation is postfixed and contains every postfixed relation. Exact finite support transport supplies either a coupling or a cut at each step. $\square$

The naive implementation may make $O((|P||Q|)^2)$ transport calls over repeated full passes. A dependency-indexed work queue and support-signature partitioning are the natural production improvement. The checker is simpler: it verifies the supplied joint matrices or replays the deletion sequence and rational deficits.

6.3 Why the negative result must not mean trace inequivalence

Consider the following two chains. In the first,

$$s \longrightarrow \tfrac{1}{2}u + \tfrac{1}{2}v, \quad u \longrightarrow a, \quad v \longrightarrow b;$$

in the second,

$$t \longrightarrow w, \quad w \longrightarrow \tfrac{1}{2}a' + \tfrac{1}{2}b'.$$

The four terminal states are absorbing. Observations are

$$o(s) = o(t) = 0, \quad o(u) = o(v) = o(w) = 1, \quad o(a) = o(a') = 2, \quad o(b) = o(b') = 3.$$

Both initial states have exactly the same observable trace distribution: after observations 0, 1, the observation is forever 2 or forever 3, each with probability 1/2. Nevertheless neither u nor v can be strongly bisimilar to w , because their next observation is deterministic while w splits between two different observations. Therefore s and t are not strongly bisimilar.

This is a unit-test control, not a merely philosophical caution. For a source goal asking for trace equality, a checked negative bisimulation result should route to Forge's existing weighted-word equality lane after the appropriate trace encoding; it must not terminate the source proof search with a refutation. That interaction adds a useful routing rule while crediting the existing closure worker rather than implementing it again [3].

6.4 Relational certificates as synthesizer witnesses

The positive relation has an operational interpretation: for each pair of related states it specifies how to correlate their next random choices. It can therefore supply an explicit coupled simulator, rather than only a Boolean equivalence answer. Such a simulator is a constructive witness for a probabilistic relational specification.

For Leant/Djex integration, the program candidate and its source-owned term remain the subject of the original type check. A separately proved reflection theorem exposes finite transition tables; only then is this worker applicable. The new artifact attached to the candidate is a distributional behavior certificate, not another typing receipt. Two distinct candidates must not inherit one another's certificate merely because their pretty-printed types coincide. This application uses the neighboring projects' existing ownership discipline rather than claiming to invent it [6, 7, 5].

7 Worker V: all-horizon coupling and perturbation bounds

Status. All-pairs affine-distance certificates and exact obstructions to the requested one-step rate are implemented for finite rational metrics.

7.1 The local quantitative obligation

Let d be a metric on a finite state space S , and let P, Q be kernels on it. Fix rational $0 \leq \kappa < 1$ and $\varepsilon \geq 0$. For every pair (s, t) , supply a coupling $\gamma^{s,t}$ of $P(s, \cdot)$ and $Q(t, \cdot)$ satisfying

$$\sum_{x,y} \gamma_{xy}^{s,t} d(x, y) \leq \kappa d(s, t) + \varepsilon. \quad (11)$$

The checker verifies all metric axioms, including the triangle inequalities, rather than trusting a field named `distance`. It then verifies the marginal constraints and bound for every supplied pair. The affine recurrence theorem itself only needs a nonnegative distance-like function; requiring a genuine metric makes the advertised transport interpretation precise.

Theorem 7.1 (All-horizon affine coupling). *Suppose (11) holds for all pairs. From any initial coupling η_0 of μ, ν , one can construct a coupling η_k of $\mu P^k, \nu Q^k$ such that*

$$\mathbb{E}_{\eta_k} d \leq \kappa^k \mathbb{E}_{\eta_0} d + \varepsilon \sum_{j=0}^{k-1} \kappa^j = \kappa^k \mathbb{E}_{\eta_0} d + \varepsilon \frac{1 - \kappa^k}{1 - \kappa}.$$

Consequently the same inequality holds for W_d after choosing an optimal initial coupling.

Proof. Define the next joint distribution by finite mixture:

$$\eta_{k+1}(x, y) = \sum_{s,t} \eta_k(s, t) \gamma_{xy}^{s,t}.$$

Summing over either coordinate and using the local marginal equations gives the required next marginals. Multiplying (11) by $\eta_k(s, t)$ and summing gives $D_{k+1} \leq \kappa D_k + \varepsilon$. Elementary induction solves this recurrence. The construction works for every finite k and does not require constructing an infinite product probability space first. $\square$

This is the finite, certificate-oriented specialization of expected-sensitivity and coupling reasoning, not a new coupling theorem [10, 9].

7.2 A worked perturbation family

Let

$$P(s, \cdot) = \kappa \delta_s + (1 - \kappa) \rho, \quad Q(t, \cdot) = \kappa \delta_t + (1 - \kappa) \sigma$$

with the discrete metric. Couple the retained states together and optimally couple the reset laws. Then (11) holds with

$$\varepsilon = (1 - \kappa) \text{TV}(\rho, \sigma).$$

For $\kappa = 1/3$, $\rho = (1/2, 1/2)$, and $\sigma = (5/8, 3/8)$, this gives $\varepsilon = 1/12$ and

$$W_d(\delta_s P^k, \delta_t Q^k) \leq 3^{-k} d(s, t) + \frac{1}{8}(1 - 3^{-k}).$$

For any L -Lipschitz observable f , the difference of expectations is at most L times this bound, by applying $|f(x) - f(y)| \leq Ld(x, y)$ to the joint distribution. The certificate can thus generate scalar inequalities for Forge’s arithmetic engine, rather than leave a transport bound unusable by the surrounding proof.

7.3 What an exact negative answer means

The producer solves the finite optimal-transport problem for each pair. If its optimum exceeds $\kappa d(s, t) + \varepsilon$, the joint matrix and dual potentials certify that *no one-step coupling for that pair meets that named bound*. This is a genuine refutation of the proposed local certificate obligation.

It is not a proof that either chain fails to mix. For example, a reset chain with retention probability $1/2$ fails a requested discrete-metric contraction rate $1/4$, but still contracts at rate $1/2$. A different metric, a multi-step kernel, or an additive slack can also succeed. The checker therefore returns a claim explicitly scoped to the supplied metric, rate, and error; it never returns a global “nonmixing” label.

7.4 Where to search next after a rate obstruction

A concrete next proposal is to try P^m, Q^m for small m and produce a certificate for the sampled process. Any claim about intermediate times then requires separately proved bounds for the remaining $0, \dots, m - 1$ steps. Another proposal fixes a finite parameterization of a metric and alternates between metric fitting and exact coupling reconstruction. Because metric coefficients and joint masses interact bilinearly, this must not be presented as one linear program unless one side is fixed.

Neither extension is part of the measured prototype. Their value here is to prescribe a sound continuation after a narrowly scoped negative answer, not to inflate the implemented capability list.

8 Worker VI: symbolic potentials for infinite-state walks

Status. Exact polynomial synthesis and independent polynomial checking are implemented for stopped, bounded-jump, skip-free random walks on $\mathbb{N}$. The infinite-horizon theorem is proved here on paper, not in Lean.

8.1 A fragment with an explicit synthesis theorem

Let J have finite support in $\{-1, 0, 1, \dots, M\}$ and rational probabilities p_j . From a positive state n , update to $n + J$; at zero, remain at zero and charge no more cost. The restriction $J \geq -1$ ensures that the positive-state transition stays in $\mathbb{N}$ without truncation. Let

$$\mu = \mathbb{E}J < 0, \quad \mathcal{L}V(n) = V(n) - \sum_j p_j V(n + j).$$

Each sample has the specified law conditional on the current history. Reusing a coin sampled once at the beginning generally does not have this semantics and must not be reflected as fresh sampling.

Given a polynomial cost $c \in \mathbb{Q}[n]$ of degree at most m , seek a polynomial V of degree at most $m + 1$, normalized by $V(0) = 0$, satisfying the discrete Poisson equation

$$\mathcal{L}V = c. \tag{12}$$

Theorem 8.1 (Triangular polynomial synthesis). *If $\mu \neq 0$, the linear map $\mathcal{L}$ is a bijection from polynomials of degree at most $m + 1$ with zero constant term to polynomials of degree at most m . In particular (12) has a unique rational solution in this normalized space.*

Proof. For $k \geq 1$, the binomial expansion gives

$$\mathcal{L}(n^k) = - \sum_{i=0}^{k-1} \binom{k}{i} \mathbb{E}[J^{k-i}] n^i.$$

The coefficient of n^{k-1} is $-k\mu$. In the ordered bases $(n, n^2, \dots, n^{m+1})$ and $(1, n, \dots, n^m)$, the matrix is triangular with nonzero diagonal $-\mu, -2\mu, \dots, -(m+1)\mu$. It is therefore invertible over $\mathbb{Q}$. $\square$

This proves completeness of the *polynomial identity solve*, not completeness of a nonnegativity prover. The producer additionally requires negative drift and nonnegative coefficients of both the resulting V and the input cost. A true expected-cost bound may be missed because of this restricted positivity vocabulary.

8.2 The accepted certificate can be weaker than equality

The checker accepts a rational polynomial V and $\varepsilon \geq 0$ when

$$V(0) = 0, \quad V(n) \geq 0 \ (n \geq 0), \quad c(n) \geq \varepsilon \ (n \geq 1), \quad \mathcal{L}V(n) \geq c(n) \ (n \geq 1). \tag{13}$$

The actual implementation proves these signs by coefficient tests: coefficients of $V(n)$ must be nonnegative; coefficients of $c(m+1) - \varepsilon$ and $(\mathcal{L}V - c)(m+1)$ must be nonnegative. This is a simple sufficient condition on the appropriate half-lines. The checker computes substitutions by Horner composition, independently of the producer's binomial-moment matrix.

Theorem 8.2 (Infinite-state cost certificate). *Under the stated stopped-walk semantics, (13) implies*

$$\mathbb{E}_n \sum_{t < T_0} c(X_t) \leq V(n).$$

If $\varepsilon > 0$, it also implies $\mathbb{E}_n T_0 \leq V(n)/\varepsilon$ and almost-sure termination.

Proof. For every fixed horizon k , the walk starting at n has finite support, bounded above by $n + kM$. Thus every finite-prefix expectation of the polynomial potential is a finite sum. Conditional use of the drift inequality and telescoping give

$$\mathbb{E}_n \left[\sum_{t < k \wedge T_0} c(X_t) + V(X_{k \wedge T_0}) \right] \leq V(n).$$

Nonnegativity permits dropping the final term. Monotone convergence of the nonnegative partial cost sums yields the total-cost bound without any prior integrability or termination assumption. The positive lower cost bound yields the expected-time conclusion exactly as in Theorem 4.1. $\square$

The checker theorem does not require a negative mean: it only requires a valid potential with the displayed inequalities. Negative mean is a restriction of this *producer*, which makes synthesis predictable. Separating search hypotheses from checker hypotheses is useful here: a later producer may find other sound potentials without changing acceptance.

8.3 Two exact examples and an essential limitation

For $\mathbb{P}(J = -1) = 2/3$ and $\mathbb{P}(J = 1) = 1/3$, we have $\mu = -1/3$ and $\mathbb{E}J^2 = 1$. Therefore

$$\mathcal{L}(n) = \frac{1}{3}, \quad \mathcal{L}(n^2) = \frac{2}{3}n - 1.$$

The unit-cost solution is $V(n) = 3n$. For cost $c(n) = n$, the unique normalized quadratic solution is

$$V(n) = \frac{3}{2}n^2 + \frac{9}{2}n, \quad \mathcal{L}V(n) = n.$$

Since $n \geq 1$ while running, the second certificate also gives $\mathbb{E}T_0 \leq V(n)$, although the separate unit-cost certificate gives the sharper $3n$. The controller should preserve both facts with their proper costs rather than confusing a total resource bound with the best time bound.

Even when (12) is exact, the theorem proved above gives an upper bound, not automatically equality of total expected cost. Finite-prefix telescoping retains a terminal expectation $\mathbb{E}V(X_{k \wedge T_0})$; dropping it is safe, but claiming equality in the limit requires proving that this remainder tends to zero. That additional uniform-integrability or tail argument is not part of the implemented checker. The recorded finite-distribution tests verify the stronger *finite-prefix* identity, not the missing limit theorem.

8.4 The fair-walk control

When the up and down probabilities are $1/2$, the producer returns UNKNOWN because the negative-drift fragment does not apply. This is not a proof of nontermination. Starting from $n > 0$, the symmetric walk reaches zero almost surely, but has infinite expected time.

These facts can be seen without appealing to a simulation: first stop on hitting 0 or $N > n$. The finite interval has absorption probability $1 - n/N$ at zero and expected exit time $n(N - n)$, obtained by solving the finite harmonic and Poisson recurrences with zero/one boundary data. The probability of eventually hitting zero is at least $1 - n/N$ for every N , hence is one. Its expected hitting time is at least $n(N - n)$ for every N , hence is infinite. Almost-sure termination and finite expected termination time are different source goals.

The negative-drift restriction is therefore not an arbitrary tuning parameter. A synthesis mode intended to prove almost-sure termination needs a different certificate language to cover this example. Increasing the current degree cap does not change what claim the worker is designed to establish.

8.5 Why Forge’s invariant-ideal machinery cannot be copied unchanged

A deterministic polynomial substitution $f \mapsto f \circ T$ is a ring homomorphism. Conditional expectation through a randomized transition is generally only linear. For a transition $X' = X$ or $X' = -X$, each with probability $1/2$,

$$\mathbb{E}[X' \mid X] = 0, \quad \mathbb{E}[(X')^2 \mid X] = X^2.$$

Thus vanishing expected observations are not closed under arbitrary multiplication. Replacing a pathwise pullback with expectation inside Forge’s ideal-closure proof rule would be unsound.

The implementation design should use different interfaces: `PathwisePullback` may expose multiplication-preservation; `MomentTransfer` exposes linearity, positivity, and proved moment equations, but no homomorphism field. This distinction is a substantive new interaction with Forge’s existing algebraic engine. Finite moment spaces can still be closed under the *linear* transfer operator when justified, while statewise polynomial nonnegativity can still be delegated to the existing cone workers.

For multivariate polynomial transitions with finite noise support, the next practical extension is to compute $\sum_j p_j V(T_j(x))$ symbolically and generate a polynomial drift obligation. Forge’s existing positivity workers may prove it over a supplied invariant domain. Completeness is not inherited from the univariate triangular theorem: multivariate operators can preserve degree, have nontrivial kernels, or require richer potentials. This route is specified, not benchmarked in the package.

9 Lean integration: concrete modules and acceptance obligations

9.1 A small semantic library, not a solver embedded in the kernel

The production goal is a Lean theorem about the *original* program, with search-free replay of the returned finite artifact. A Python Boolean verdict is not such a theorem. The proposed module structure is

ForgeQ/RationalModel	-- exact tables; normalization; stopped semantics
ForgeQ/FiniteHorizon	-- expectation monotonicity; prefix bounds
ForgeQ/Reachability	-- upper bound + policy + exit-progress theorem
ForgeQ/Cost	-- nonnegative total-cost bridge
ForgeQ/Coupling	-- marginals; support; mixture; dual inequalities
ForgeQ/Bisimulation	-- greatest relation; sequential Hall deletions
ForgeQ/Distance	-- all-horizon affine recurrence and transport bound
ForgeQ/PolynomialDrift	-- reified polynomial transfer and sign obligations
ForgeQ/Reify	-- source-specific model equations
ForgeQ/Replay	-- finite checker/proof reconstruction
ForgeQ/Tactic	-- goal recognizers and calls into existing Forge

These are proposed modules, not a claim that they exist in Forge today. The archive supplies only `Monotone.lean` and `FiniteHorizon.lean` as candidate foundations. The latter contains

rational weighted-sum monotonicity, a fixed-kernel finite-prefix cost bound, and a coupling-marginal expectation lemma. It does *not* contain an all-scheduler theorem, a PMF bridge, a complete checker, or an infinite-horizon theorem.

9.2 Concrete library reuse

Mathlib already represents discrete probability laws through **PMF**, a function into nonnegative extended reals whose sum is one. Its **Basic** module provides conversion to a probability measure; **Monad** provides **pure**, **bind**, and their equations; **Constructions** provides finite constructors and **map**; **Integrals** connects finite PMF expectations to weighted sums [13, 14, 15, 16]. These are suitable source semantics, not a replacement for the exact rational checker.

Library / facility	Concrete use	Required caution
<code>PMF.ofFintype</code> , <code>PMF.ofFinset</code>	Construct source laws from proved finite masses	Cast rational nonnegativity and normalization explicitly
<code>PMF.bind</code> , <code>PMF.bind_apply</code>	Interpret sequential sampling and coupled steps	Fresh conditional law, not just an unconditional marginal
<code>PMF.map</code> , <code>PMF.map_bind</code>	Transport state encodings and observations	Preserve the actual output map
<code>PMF.integral_eq_sum</code>	Convert finite expectations to rational sums after casting	Infinite-support integrals require separate hypotheses
<code>Finset.sum</code> and ordered-field lemmas	Marginals, weak duality, and Bellman inequalities	Do not introduce numerical tolerances
<code>norm_num</code> , rational arithmetic, existing Forge polynomial replay	Discharge exact closed arithmetic and polynomial identities	Kernel-checked terms under the existing trust profile
Storm, optional external search	Propose finite-model values, policies, or exact answers	Reconstruct this package's certificates; never trust a printed result

The Mathlib API names above were checked against current generated documentation. The supplied Lake configuration pins Forge's recorded Mathlib revision and Lean v4.34.0; those files were not built here. Documentation availability is not compilation evidence against the pin.

Storm's official guide documents explicit, PRISM, and JANI input, reachability and reward queries, and the `-exact` flag [17]. For example, an external experiment can use

```
storm --prism model.pm --prop 'Pmax=? [F "target"]' --exact
```

This is a concrete optional search backend, not an executed dependency of the delivered prototype. A production adapter must obtain the full table and selected policy, then compute boundary sets and exit ranks or other certificates itself. A single initial-state number is insufficient for the all-state Bellman proof. Pin and record the actual Storm version when that adapter is built; this article does not invent a tested version or an export API.

9.3 Source reflection must expose equations, not just tables

For an encoded finite source type S_0 and an enumeration $e : \text{Fin}(n) \simeq S_0$, a bridge must prove the source transition masses equal the reflected entries under e . The stopping predicate, cost, and observations require their own equalities. An MDP additionally requires completeness of each enabled action enumeration. Omitting an action invalidates a universal bound even when all listed actions pass.

A useful interface, shown as a specification rather than compiled code, is

```
reflectFinite sourceGoal :
  (problem : RationalProblem) *
  (bridge : SemanticClaim problem -> OriginalGoal)

replay problem certificate :
  Either Rejection (proof : SemanticClaim problem)
```

The certificate worker has no authority to construct **bridge**. Reflection produces that proof in Lean. A first implementation should accept only source constructors whose equations are already known: finite distributions, explicit finite kernels, mapped kernels, and a small reflected loop language. Arbitrary higher-order Lean functions should remain opaque unless an appropriate reflection theorem is supplied.

A randomized loop should not be given a total output PMF by fiat. It can diverge. Use nonnegative subdistribution semantics or finite-prefix semantics until mass one or almost-sure termination has been proved. Normalizing a deficient output mass changes the program to its conditional-on-termination distribution. That is not a harmless simplification and cannot justify an unconditional source theorem.

9.4 Rational versus real proof reconstruction

Perform certificate arithmetic over $\mathbb{Q}$, with normalized integer numerator and positive denominator representations. Prove the rational equalities/inequalities first; cast them to the real or extended-real semantic domain using nonnegativity and finite-sum lemmas. This separates exact computation from analytic limits.

For the first end-to-end milestone, explicit proof reconstruction from finite rows is preferable to a large reflected Boolean interpreter: it yields a smaller theorem statement and exposes normalization bugs. Once the theorem families are stable, define finite checkers with soundness theorems. Large tables may then be checked by ordinary kernel reduction or by carefully proved reflection. A native evaluator result alone must not replace a kernel proof under Forge’s strict trust policy. The existing repository already discusses those trust profiles; they are prerequisites, not new proposals here [5].

The candidate `costPrefix_le` theorem illustrates the small core. If P is nonnegative, $V \geq 0$, and $c + PV \leq V$, induction proves $C_k \leq V$ where $C_{k+1} = c + PC_k$. Row normalization is not needed for this algebraic lemma; it is needed by the interpretation of P as probabilities. Keeping these two obligations separate makes the theorem reusable without hiding a missing semantic premise.

9.5 How the workers cooperate with Forge and Leant

The controller should recognize a quantitative head symbol only after elaboration and source normalization. It then routes according to the requested proposition: reachability maximum; universal cost; supported coupling; strong bisimulation; a metric bound; or a symbolic drift obligation. Plain trace equality should continue to use the already-developed weighted-word lane when applicable.

A concrete synthesis pipeline for a randomized finite DSL is: Leant/Djex proposes a typed candidate using its existing provider inventory; Lean checks the original candidate type; the DSL reflection theorem yields its finite kernel; the new workers establish output behavior or a cost bound; and the controller attaches the resulting theorem to that exact candidate. A type such as a distribution-valued function does not itself express unbiasedness, equivalence, or expected runtime. Those are new proof obligations, not inferred properties of successful type inhabitation [6, 7].

Do not add probabilistic reasoning to Djex’s complete propositional inhabitation fragment by simply erasing probabilities. Such an abstraction can guide candidate ranking, but its negative result cannot refute a probabilistic behavioral specification. A failed strong-bisimulation obligation likewise cannot prune a candidate required only to preserve trace laws. A rate obstruction may request another metric or block size. These semantic ceilings should be represented by different result constructors, not by comments attached to one generic `false` value.

9.6 Integration gates that would establish a real tactic gain

The first gate is a source theorem for a two-state retry loop: from a source PMF transition, a produced rational potential, and no handwritten value lemma, reconstruct the finite-prefix bound and then the expected-duration theorem in Lean. Record the exact source, generated proof, dependency pin, and axiom inventory. This gate is small enough to diagnose the whole boundary.

The second gate is the three-state success/failure example from Section 3. Require the exact probability $1/2$ while rejecting an attempt to present its exit rank as a finite success-time bound. This tests the least-solution argument, not just rational arithmetic.

The third gate is an original pair of reflected randomized programs: construct a joint witness, prove marginals in Lean, and derive the actual source relation. Include the trace-equivalent/non-bisimilar control and test that the controller does not issue a source-level refutation.

Only after these gates should the project claim a native quantitative extension. Compare against direct `grind`, an ordinary tactic portfolio, and the same portfolio with the general quantitative lemmas available but without automated certificate search. That last comparison isolates the contribution of *finding* the witness from the contribution of adding a new library theorem. No such Lean comparison was run for this article.

10 Implementation, formats, and resource accounting

10.1 Executable separation between producer and checker

The Python package contains two deliberately separate application modules. `forgeq.search` implements exact Gaussian elimination, policy enumeration, residual-network search, relation refinement, and polynomial coefficient solving. `forgeq.checker` imports none of them. It parses

the external problem and artifact, checks finite rational obligations, and returns an outcome naming the claim actually supported.

The independent oracle module is test-only. Replay installs an import guard forbidding the producer, the oracle, and NumPy/SciPy/SymPy, then checks the saved artifacts under `python -S`. This is stronger than merely choosing not to call a producer function during the recorded run. It is still not formal verification: the modules share Python and its standard-library rational arithmetic, and independently written code can contain correlated mistakes.

The reference producers expect well-formed dictionaries. They are not sandboxed services and do not themselves provide a complete hostile-input boundary. The checker validates the original problem, not just the artifact. A production controller must impose process, input, and arithmetic budgets before starting any producer.

10.2 Concrete certificate schemas

The following are field-level descriptions, not an alternative permissive serialization:

```
reach_max:
  value[n], policy[n], dead[index-set], rank[n]
runtime_upper:
  potential[n]
transport:
  joint[n][m], left_potential[n], right_potential[m]
hall:
  left_set[index-set], neighbors[index-set]
bisimulation:
  relation[pair-set], couplings[{pair, joint}]
no_bisimulation:
  removals[{pair, cut: hall}]
contraction:
  joints[n*n][n][n] # outer order is lexicographic state pairs
rate_obstruction:
  pair, transport: primal-dual certificate
polynomial_cost:
  potential[constant-first coefficients], epsilon
unknown:
  reason # not accepted by any semantic checker
```

Every semantic certificate also has its exact `kind` field. Unknown is a search result, not a proof object. The closed schemas reject unknown fields, malformed dimensions, duplicate indices, and out-of-range identifiers. JSON parsing rejects duplicate keys, floating-point numbers, NaN, and infinity. Rational values must be exact integers or canonical rational strings, with a positive nonzero denominator and bounded numerator/denominator bit lengths. Booleans are rejected as numeric values despite Python’s integer-subclass relationship.

Current checker limits include 48 states, 16 actions per state, a 4096-bit rational bound, a polynomial degree cap, and a 32 MB JSON-text cap. These are conservative admission limits, not mathematical completeness bounds. The parser is not advertised as fully hardened against hostile nesting, arithmetic denial of service, or every Python object-injection scenario. Errors and budget exhaustion must remain operational failures.

10.3 Search costs versus checking costs

The most expensive routine here is often all-pairs transport, not arithmetic replay. The contraction certificate can contain n^4 joint entries in a dense representation, even though each local transport problem is small. Sparse joints with explicit dimensions and zero defaults are an obvious extension, provided omitted entries have exactly defined semantics.

The reachability checker is linear in the encoded transition size plus its vectors. Positive bisimulation checking is proportional to the total joint data. A naive negative checker may scan the whole current relation for every Hall cut. The prototype prioritizes transparent replay over optimized adjacency indexes. Polynomial checking uses exact Horner shifts and coefficient comparison; its field-operation complexity is polynomial in degree and jump support, while bit sizes can grow.

All rational-operation counts must be separated from bit complexity. Denominator growth can dominate even when a matrix dimension is tiny. A production scheduler should record maximum rational bit size, certificate size, number of policy trials, number of flow augmentations, and proof-reconstruction work. This package records wall-clock producer/checker times and artifact sizes but does not yet expose every internal search counter. Those missing counters are an instrumentation task, not a measured performance claim.

11 Recorded experiments and their limits

11.1 Protocol and independent computations

The final run used Python 3.13.5 on Linux, seed 20260915, and only the standard library. Each record stores the original problem, complete certificate, checker outcome, local timings, and serialized certificate size. The seeded corpus contains 520 records and 508 distinct worker/problem pairs. Duplicate generated problems are retained and counted explicitly; the records are not 520 distinct theorems.

The independent computations are intentionally different from search. Reachability is compared against vertices of a bounded Bellman-supersolution polytope, solved by determinants and Cramer’s rule rather than policy enumeration and Gaussian elimination. Uniform runtime is compared against exact backward formulas on forward-with-self-loop MDPs. Transport is compared against exhaustive small integer contingency tables. Bisimulation is compared against probability-to-block partition refinement on the disjoint union, with no coupling computation. Discrete-metric contraction is compared with total variation, and symbolic costs are checked by explicit finite distribution evolution and a finite-prefix telescoping identity.

The last oracle has a different evidentiary scope from an exact finite decision oracle: checking 12 horizons does not independently prove an infinite-horizon bound. That claim rests on the certificate conditions and Theorem 8.2. The finite-horizon oracle is a cross-check of the transition and polynomial computations. There are 600 such prefix checks across the 50 symbolic records; they are not added to the record total.

There are 60 accepted reachability certificates, with 53 records containing at least one strictly fractional state value. All 80 negative bisimulation records have matching start observations and a nonempty deletion trace; they are not immediate label-mismatch rejections. The 40 positive bisimulation cases are generated by relabeling states of an existing chain, so they should not be interpreted as a difficult semantic-equivalence benchmark.

Worker	Records	Distinct	Oracle matches	Mutations rejected
Maximum reachability	60	57	60	60
Uniform runtime	50	49	50	50
Transport / Hall	180	173	180	180
Strong bisimulation	120	120	120	120
Metric bound / obstruction	60	59	60	60
Symbolic polynomial cost	50	50	50	50

Table 3: Final corpus. Counts are experiment records, not Lean theorems.

The transport outcomes include both feasible optima and valid support obstructions. The contraction negatives refute requested rates, not the chains’ convergence. A successful experiment is agreement with the stated scoped outcome, not necessarily a positive theorem of the most ambitious kind.

11.2 A benchmark correction made during development

The initial pilot used dense reachability models without a rejecting sink; many cases therefore collapsed to the uninformative value one. It also allowed immediate observation-mismatch bisimulation failures and repeated a small set of polynomial instances. The final generator adds an absorbing rejecting state, forces matching initial observations in negative bisimulation cases, and varies holding probabilities and polynomial costs. The pilot summary remains as `results/pilot-summary.json`; it is not combined with the final counts. The final raw problems and artifacts, rather than the pilot, are the replay corpus.

This correction matters more than adding another hundred easy instances. Even the revised cases remain very small: reachability has three or four states, runtime and bisimulation use at most five states, transport marginals have at most three entries, contraction has at most five states, and symbolic costs have degree at most five. The symbolic corpus uses jumps $-1, 0, 1$; larger upward jumps supported by the code are not a broad tested workload here.

11.3 Mutation tests, unit tests, and inconclusive controls

There is one deliberately invalid mutation per recorded certificate, and all 520 are rejected. Mutations include setting a target value to zero, erasing a unit-cost potential, changing a joint mass by 10^{-30} , replacing a Hall deficit by an empty cut, omitting a positive coupling obligation, inserting an invalid deletion index, and changing a symbolic potential’s boundary value. These test different obligations but are not a security proof or comprehensive mutation-coverage study. Arbitrary mutations of an upper bound need not be invalid; the generator deliberately avoids counting harmless weakening as a checker failure.

Separately, 24 unit-test methods pass both normally and with Python optimization enabled. Some methods contain loops over several examples, so a method count is not a mathematical case count. Controls cover the spurious harmonic self-loop, absent progress, scheduler polarity, malformed probability laws, unsafe negative jumps, exact residuals, invalid metrics, source substitution, zero cost, and the trace-equivalent/non-bisimilar pair. For the latter, explicit observation-prefix distributions agree for lengths one through ten, while the paper’s simple trace argument covers all lengths.

Five stored searches return UNKNOWN: zero policy budget, a model lacking a finite uniform runtime certificate, the fair-walk fragment exclusion, zero polynomial degree budget, and zero flow-augmentation budget. None is admitted as a source-level refutation. Their distinct reasons remain in `unknown-controls.json`.

11.4 What the timing data do and do not establish

Worker	Search median (ms)	Check median (ms)	Max cert. bytes
Maximum reachability	0.126	0.144	117
Uniform runtime	0.171	0.134	74
Transport / Hall	0.199	0.046	148
Strong bisimulation	0.886	0.163	799
Metric bound / obstruction	4.903	0.677	3294
Symbolic polynomial cost	0.114	0.153	126

Table 4: Local Python timings, excluding independent-oracle time; no Lean comparison.

The table reports medians of one local run, not benchmarked speedups. Parsing, search, and checking have different costs; tiny examples can have a checker median exceeding the producer median. In particular no claim is made that certificate replay must be faster on every input. The useful architectural fact is that replay performs different, simpler obligations and runs without the producer, not a universal timing inequality.

All 520 artifacts replay successfully with producer and oracle imports forbidden, and all 520 saved mutations are again rejected. This verifies reproducibility at the Python evidence boundary. It does not establish that any certificate has been checked by Lean’s kernel.

11.5 Lean status and the remaining gap

The actual local elaboration harness recorded NOT_RUN for both the core and Mathlib lanes because `lean` and `lake` were unavailable. No external compiler was used. Consequently the supplied Lean sources are candidate code, not compiled artifacts; their proof text contains no `sorry`, but absence of that token is not evidence of successful elaboration.

The package therefore establishes neither a working native `forge` extension, nor source-to-model correctness, nor an axiom audit, nor a tactic comparison. This is the same kind of implementation gap the existing Forge conclusion correctly emphasizes [4]. The new result here is a concrete quantitative branch with independently exercised certificate mechanisms, not closure of that gap by relabeling Python checks as formal proofs.

12 Development priorities and assessment

12.1 An order that minimizes unverified surface area

First formalize rational finite sums, coupling marginals, weak duality, and finite-prefix supersolution bounds. These are small theorem families with sharply stated hypotheses and immediate utility. Integrate one source constructor and one successful retry-loop theorem before broadening the parser.

Next formalize the exit-rank reachability theorem and the sequential Hall-deletion theorem. These are the two places where a plausible local certificate can be globally misleading: fixed points need progress, and negative relation refinement needs the correct historical relation. Their adversarial examples should be permanent Lean regression tests.

Then connect finite-prefix semantics to nonnegative extended expectations, followed by the symbolic walk bridge. Only at that point should an infinite-state potential be presented as an automatically proved source theorem. Improving polynomial search before proving this bridge increases search coverage but not assurance.

Finally replace reference enumeration with production oracles, add sparse certificate forms, and integrate the workers into Forge’s existing obligation broker. Keep the exact checker and artifact formats independent of the chosen optimizer. For quantitative synthesis, build a small finite randomized DSL corpus before attempting arbitrary higher-order definitions from Leant.

12.2 Explicit non-goals of this submission

This is not general probabilistic-program verification, automatic measure-theoretic reasoning, full probabilistic temporal logic, differential privacy verification, or a complete termination prover. It does not implement interval-uncertain probabilities, continuous distributions, general nondeterministic probabilistic bisimulation, learned metric search, or multivariate supermartingale synthesis. Those require additional semantics or certificate families, not just a larger budget.

It also does not claim a new fundamental theory of coupling or MDPs. The mathematical advantage is choosing proof objects whose finite checks are sufficient for the desired quantified statements, making their precise scope executable, and showing how they coexist with the already-developed Forge mechanisms without unsoundly reusing an algebraic abstraction.

12.3 Recommendation

Add a *quantitative* branch to Forge, but make its first release narrow: finite rational source kernels, supported coupling witnesses, and universally quantified finite-prefix bounds, followed by a checked unbounded-time bridge. The strongest new capabilities in this design are exact reachability with a certifiable proper policy, proof-producing relational refinement, and all-horizon quantitative bounds. The symbolic potential worker is the most direct path from those finite foundations to infinite-state mathematics.

The right next acceptance criterion is not another larger Python count. It is an original Lean goal closed through a proved source bridge and a reconstructed certificate, with the negative semantic controls still refusing the wrong conclusions. This package provides concrete algorithms, proof obligations, reference implementations, and test data for reaching that criterion.

A Reproduction and artifact map

From the archive root, the main commands are

```
cd prototype
python -S -m unittest discover -s tests -v
python -O -S -m unittest discover -s tests -v
python -S replay.py
```

```
python -S run_experiments.py
```

The last command writes to `reproduced-results/`, leaving the recorded corpus untouched. To attempt the Lean candidates in an environment with the pinned toolchain and dependencies, use the documented commands in the archive README; the harness writes its actual outcome, including `NOT_RUN`, rather than treating a missing compiler as success.

Path	Content
<code>article/forged.tex</code> , <code>forged.pdf</code>	This article and editable source; generated tables are separate small TeX inputs.
<code>prototype/forged/search.py</code>	Untrusted exact-rational producers.
<code>prototype/forged/checker.py</code>	Independent finite certificate checks and strict parser.
<code>prototype/oracles.py</code>	Small, independently structured test oracles.
<code>prototype/tests/</code>	Unit tests, including semantic-ceiling controls.
<code>prototype/run_experiments.py</code>	Seeded corpus and mutation generator.
<code>prototype/replay.py</code>	Replay with producer/oracle imports prohibited.
<code>results/</code>	Original problems, certificates, invalid mutations, unknown controls, timings, and actual status logs.
<code>lean/</code>	Uncompiled candidate foundations and pinned Lake configuration.
<code>docs/</code>	Source audit, capability ledger, benchmark notes, integration contract.
<code>tools/</code>	Article build and actual Lean-attempt harness.

B Primary references and source pins

The repository references below identify the inspected baseline rather than an unqualified moving branch. Mathlib and Storm documentation describe available library interfaces, not dependencies executed during the experiment. All web sources were consulted on September 15, 2026. No third-party source archive, paper PDF, or font file is redistributed with this package.

References

- [1] V. Reshetnikov, *Forge: README*, revision `c98e47c5f804e92880fc1d0e378c1b95832b685c`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/README.md>.
- [2] V. Reshetnikov, *Forge: provenance and merge ledger*, same revision, `article/sections/B-provenance.tex`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/B-provenance.tex>.
- [3] V. Reshetnikov, *Forge: finite certificates for unbounded behaviour*, same revision, `article/sections/06-closure.tex`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/06-closure.tex>.
- [4] V. Reshetnikov, *Forge: assessment and recommended starting point*, same revision, `article/sections/12-conclusion.tex`. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/12-conclusion.tex>.

- [5] V. Reshetnikov, *Ideas from Leant and Djex*, same Forge revision, docs/IDEAS-FROM-LEANT-DJEX.md. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [6] V. Reshetnikov, *Leant: root README*, revision 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [7] V. Reshetnikov, *Djex: root README*, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [8] F. Funke, S. Jantsch, and C. Baier, *Farkas certificates and minimal witnesses for probabilistic reachability constraints*, arXiv:1910.10636, 2019; revised 2020. <https://arxiv.org/abs/1910.10636>.
- [9] G. Barthe, B. Grégoire, J. Hsu, and P.-Y. Strub, *Coupling proofs are probabilistic product programs*, arXiv:1607.03455, 2016; POPL 2017. <https://arxiv.org/abs/1607.03455>. DOI: <https://doi.org/10.1145/3009837.3009896>.
- [10] G. Barthe, T. Espitau, B. Grégoire, J. Hsu, and P.-Y. Strub, *Proving Expected Sensitivity of Probabilistic Programs*, arXiv:1708.02537, 2017. <https://arxiv.org/abs/1708.02537>.
- [11] A. Chakarov and S. Sankaranarayanan, *Probabilistic Program Analysis with Martingales*, CAV 2013. Author publication page: <https://plv.colorado.edu/papers/martingales-cav13.html>.
- [12] K. G. Larsen and A. Skou, *Bisimulation through probabilistic testing*, Information and Computation 94(1), 1–28, 1991. DOI: [https://doi.org/10.1016/0890-5401\(91\)90030-6](https://doi.org/10.1016/0890-5401(91)90030-6).
- [13] Mathlib contributors, *Mathlib.Probability.ProbabilityMassFunction.Basic*, generated API documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/ProbabilityMassFunction/Basic.html.
- [14] Mathlib contributors, *Mathlib.Probability.ProbabilityMassFunction.Monad*, generated API documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/ProbabilityMassFunction/Monad.html.
- [15] Mathlib contributors, *Mathlib.Probability.ProbabilityMassFunction.Constructions*, generated API documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/ProbabilityMassFunction/Constructions.html.
- [16] Mathlib contributors, *Mathlib.Probability.ProbabilityMassFunction.Integrals*, generated API documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/ProbabilityMassFunction/Integrals.html.
- [17] Storm developers, *Running Storm*, official user guide. <https://www.stormchecker.org/documentation/usage/running-storm.html>.